KLAi

Five small builds, from linear regression to attention, written from first principles

Can five small builds, written from first principles, reproduce the core of what runs inside modern AI?

Mathematics for AI and Machine Learning | Session 18 of 18 | Course project

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Build linear regression (normal equation and gradient descent) and logistic regression (gradient descent) with numpy

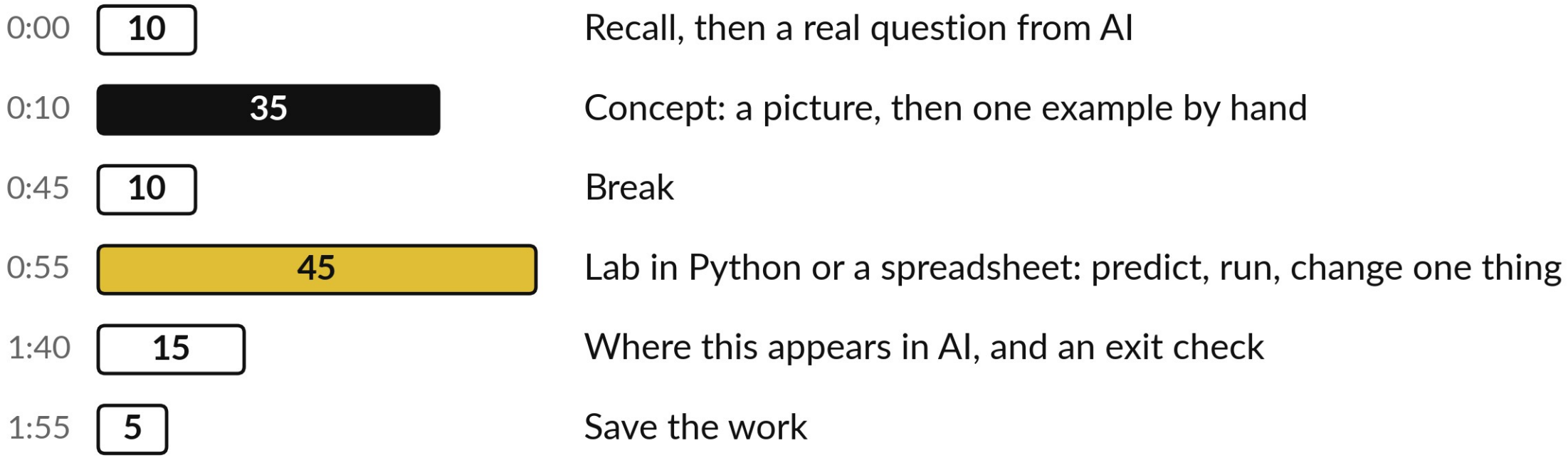


Build PCA from the covariance eigen-decomposition, and a 2-4-1 neural network with backpropagation (take-home)



Learn attention as a ladder of known steps, build it for three tokens, and see OK on the self-checks of your project

Your two hours: a guided tour, attention, then your project



The tour first. Then your project, with self-checks and no marks. Then AI.

Three questions from last time

1. How are cross-entropy, entropy and KL divergence related?

2. What is the gradient of softmax with cross-entropy with respect to the logits?

3. What does a perplexity of 5 mean?

Answer aloud first. Then we check together.

Answers to the three questions from Session 17

Question	Short answer
How are cross-entropy, entropy and KL divergence related?	Cross-entropy = entropy + KL divergence. The entropy is fixed by the data, so lowering the cross-entropy lowers the KL divergence.
What is the gradient of softmax with cross-entropy with respect to the logits?	The prediction minus the target: p - y.
What does a perplexity of 5 mean?	The model is as unsure as if it chose among about five equally likely words at each step.

All three come back today: p - y trains Builds 2 and 4, and the softmax is inside attention.

Words we use today

	Word	What it means
	**Logistic regression**	A model that passes a straight-line score through the sigmoid to give a probability between 0 and 1.
	**Principal component**	A direction of largest spread in the data: an eigenvector of the covariance matrix.
	**Backpropagation**	The chain rule applied backwards through a network to get the gradient of the loss for every weight.
	**Query, key, value**	The three vectors each token makes in attention: what it looks for, what it offers and what it passes on.
	**Attention**	Each token takes a weighted average of the value vectors, with weights from softmax(Q K^T / sqrt(d)).

Five builds: four reuse what you know, and attention is new today



Linear regression

Sessions 4, 10



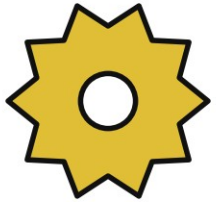
Logistic regression

Sessions 11, 17



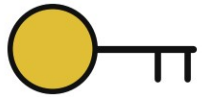
PCA

Sessions 5, 6



Neural network

Sessions 9, 11



Attention

New today

Illustrative data. Each build is 3 to 25 lines of numpy.

The illustrative data of Builds 1 to 3 (made-up numbers to show the arithmetic)

Build	Data
1 Linear	Hours x = 1, 2, 3, 4, 5, 6. Energy y (kWh) = 2.2, 2.9, 4.1, 5.2, 5.8, 7.1.
2 Logistic	Hours x = 1 to 8. Passed y = 0, 0, 0, 1, 0, 1, 1, 1.
3 PCA	a = 2.5, 0.5, 2.2, 1.9, 3.1, 2.3, 2.0, 1.0, 1.5, 1.1. b = 2.4, 0.7, 2.9, 2.2, 3.0, 2.7, 1.6, 1.1, 1.6, 0.9.

These fixed numbers are for the slides and the guided workbook. In the notebook your student ID makes your own different data.

The illustrative data of Builds 4 and 5

Build	Data
4 Network	W1 rows (0.5, −0.3), (0.2, 0.8), (−0.6, 0.4), (0.1, −0.9). b1 = 0. W2 = (0.3, −0.5, 0.7, 0.2). b2 = 0.1.
4 XOR rows	Inputs (0, 0), (0, 1), (1, 0), (1, 1) with targets 0, 1, 1, 0.
5 Attention	X rows (1, 0), (0, 1), (1, 1). Wq rows (1, 0), (1, 1). Wk rows (1, 1), (0, 1). Wv rows (1, 0), (0, 2). d_k = 2.

Q = X Wq, K = X Wk and V = X Wv give three rows of two numbers each (one row per token).

Build 1: linear regression has a direct solution and an iterative one

Normal equation

$$w = (X^T X)^{-1} X^T y$$

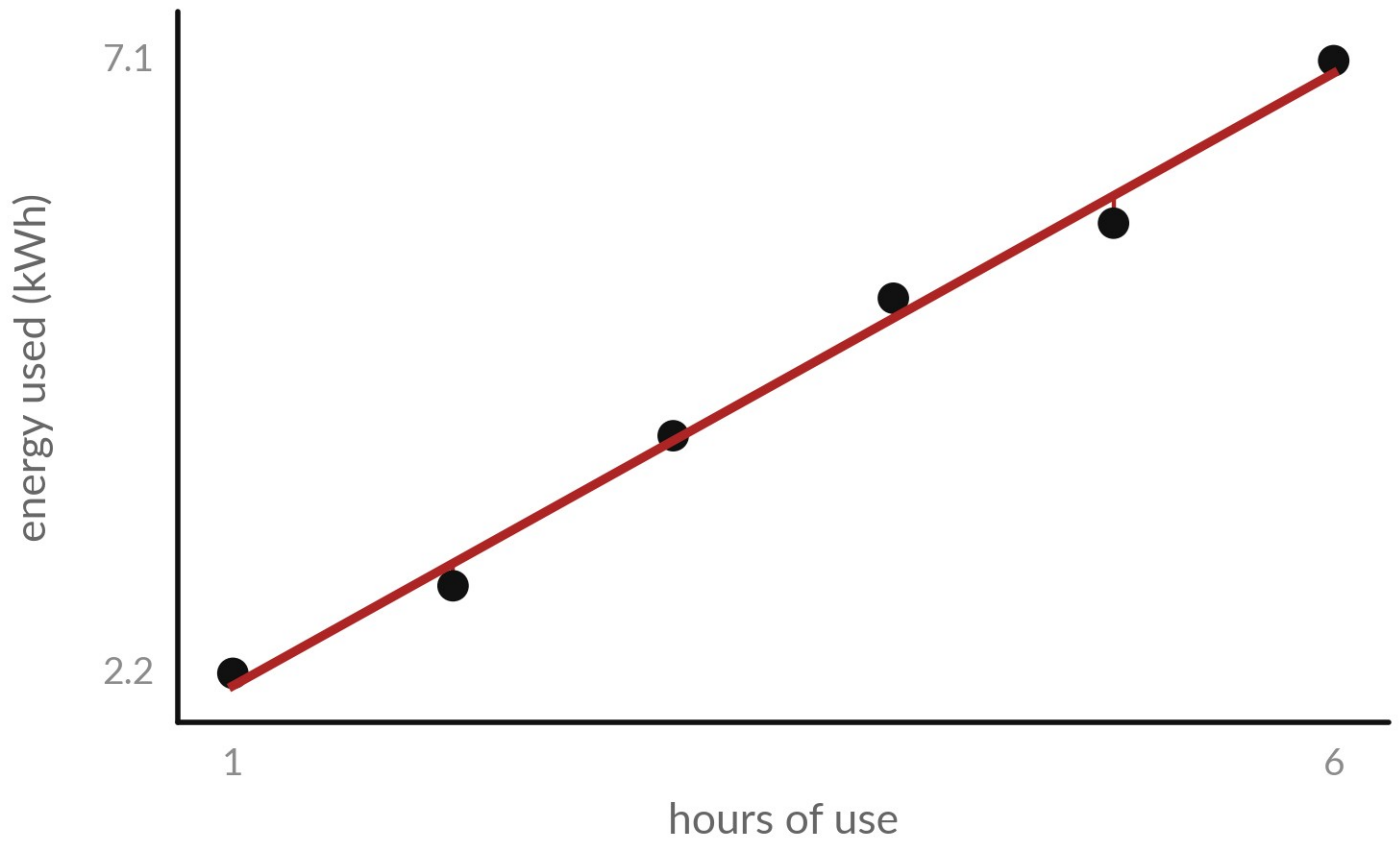
Gradient descent step

$$w \leftarrow w - \eta \frac{2}{n} X^T (Xw - y)$$

Symbol	What it means	In our data
X	The design matrix: a column of ones, then x.	6 rows by 2 columns
w	The weights: the intercept b (it multiplies the ones), then the slope w1.	(1.12, 0.98)
eta	The learning rate (lr in code).	0.02
n	The number of rows.	6

At eta 0.02, 100 steps give (0.714, 1.075); about 5,000 steps reach (1.12, 0.98). MSE 0.0213.

Build 1 in a picture: the fitted line and its residuals

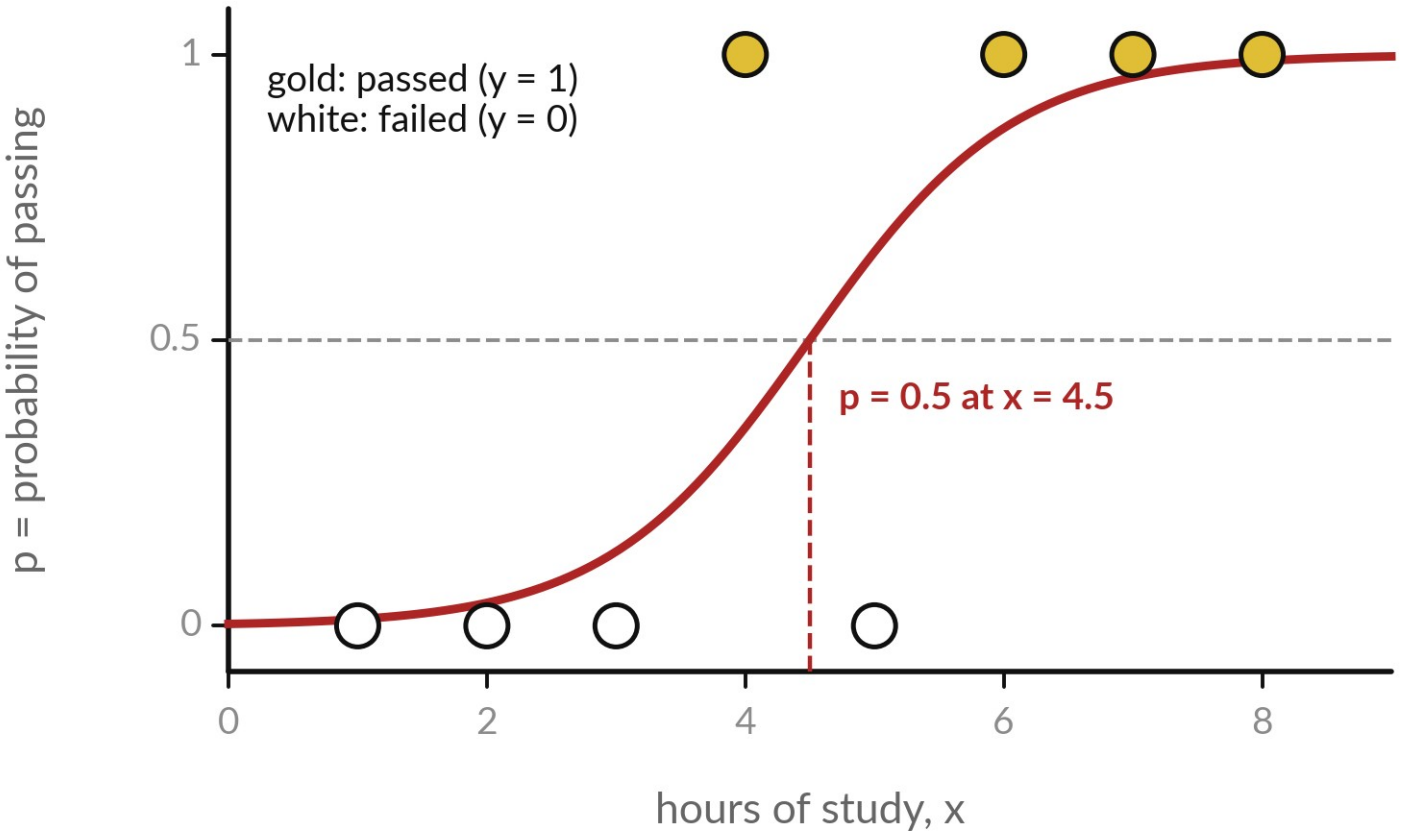


The line is y-hat = 1.12 plus 0.98 x. The red segments are the residuals.

Illustrative data: x = 1 to 6 hours, y = 2.2, 2.9, 4.1, 5.2, 5.8, 7.1 kWh.

The normal equation and enough gradient steps give the same line.

Logistic regression turns a straight-line score into a probability with the sigmoid



Logistic regression: score = b plus w1 x, then p = sigmoid(score), the probability of class 1.

The sigmoid of Session 11 squeezes any score into 0 to 1.

Trained on the 8 students: p = 0.5 at 4.5 hours. Predict "pass" above it.

Build 2: logistic regression is trained by gradient descent on the cross-entropy

Model

$$p = \sigma(b + w_1 x) = \frac{1}{1 + e^{-(b + w_1 x)}}$$

Gradient descent step

$$w \leftarrow w - \eta \frac{1}{n} X^T (p - y)$$

Symbol	What it means	In our data
p	The predicted probability of class 1 (passed).	0.5 at the start
p - y	Prediction minus target (Session 17, two classes).	one value per row
eta	The learning rate.	0.1
Loss	Binary cross-entropy, from Session 16.	0.693 at the start

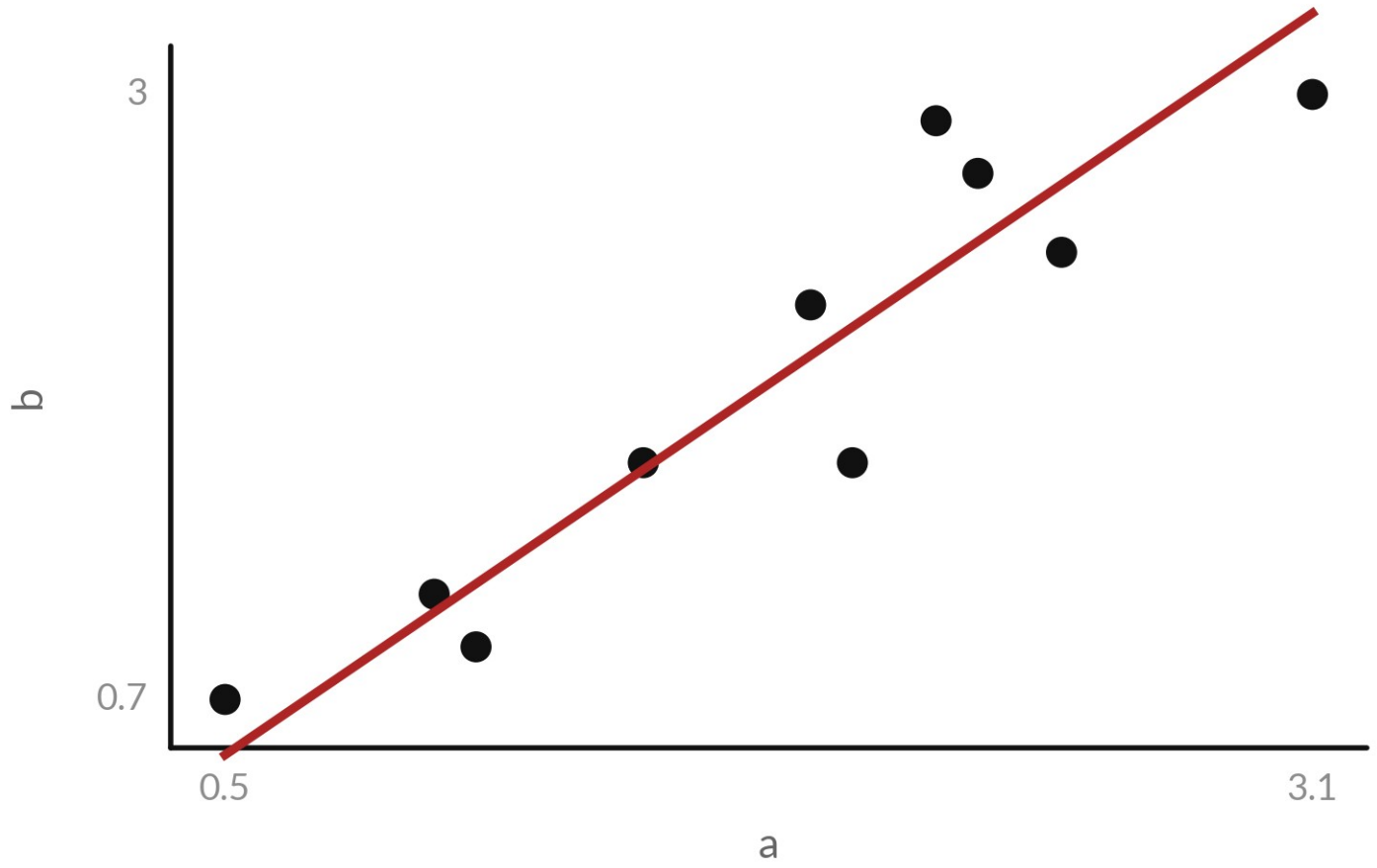
After 5 steps: w = (–0.0509, 0.1494), loss 0.623. Data: 8 students (see the data slide).

Build 2 by hand: the loss falls at every one of the first five gradient steps

Step	b	w1	Loss (nats)
0	0.0000	0.0000	0.6931
1	0.0000	0.0875	0.6407
2	−0.0096	0.1207	0.6316
3	−0.0224	0.1359	0.6281
4	−0.0364	0.1442	0.6255
5	−0.0509	0.1494	0.6232

Learning rate 0.1. Step 0 is the start. The workbook has the same table.

Build 3: PCA finds the direction of largest spread in the data



The red line is the first principal component. It holds 96.3 percent of the variance.

Eigenvalues of the sample covariance matrix: 1.284 and 0.049.

First component: (0.678, 0.735).

PCA as formulas: centre, form the covariance, find its eigenvectors, project

Sample covariance matrix

$$C = \frac{1}{n-1} X_c^T X_c$$

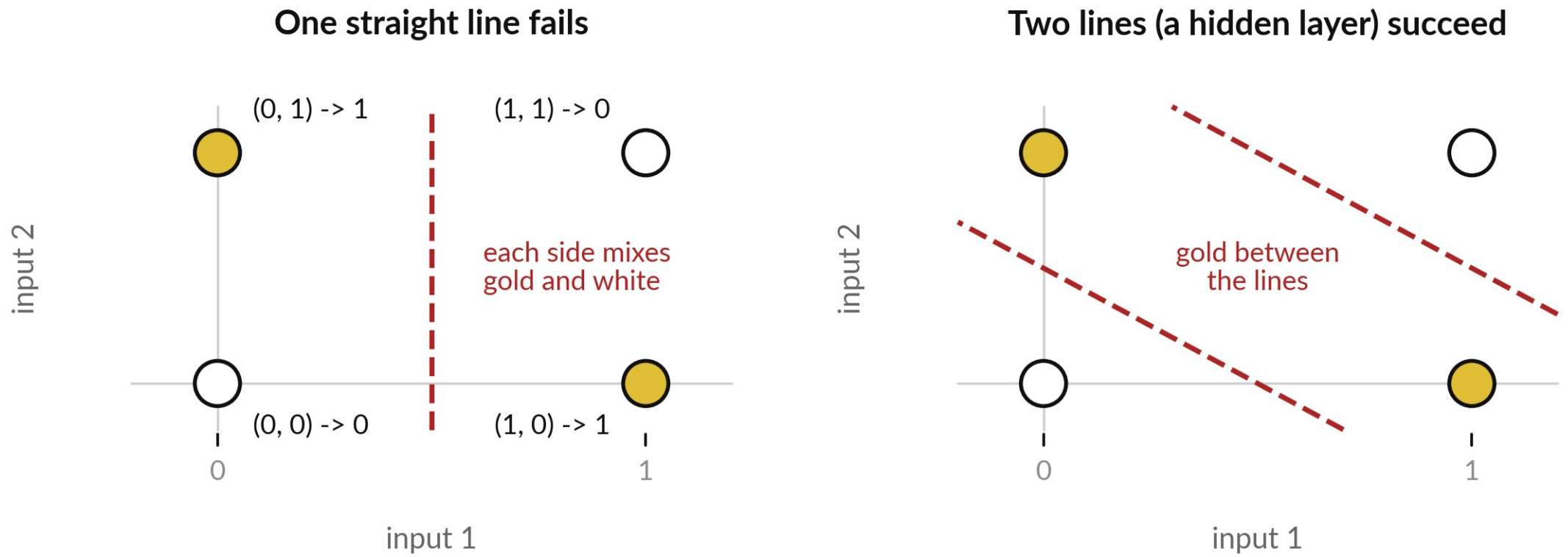
Eigenvector and score

$$Cv = \lambda v, \qquad \text{score} = X_c\, v_1$$

Symbol	What it means	In our data
X_c	The data with each column centred on its mean.	10 rows by 2 columns
lambda	An eigenvalue: the variance along its eigenvector.	1.284 and 0.049
v_1	The eigenvector with the largest eigenvalue: the first component.	(0.678, 0.735)
n - 1	Sample data divide by n - 1 (Session 5, np.cov).	9

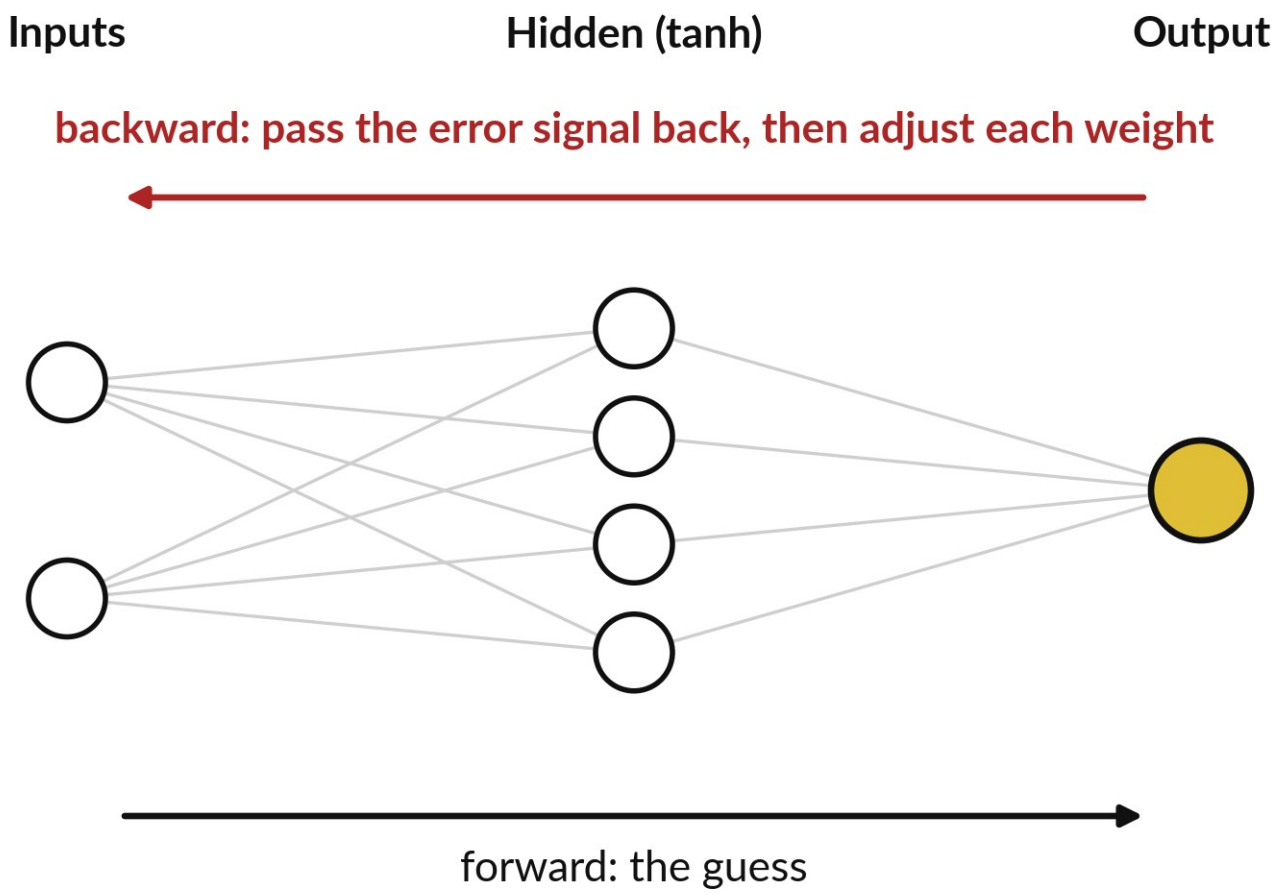
Dividing by n instead (ddof = 0) gives 1.156 and 0.044: the same eigenvectors and the same 96.3 percent share.

XOR: no single straight line separates the two classes, but two lines can



XOR: 1 (gold) when exactly one input is 1, else 0 (white).

Build 4: a 2-4-1 network has two layers of weights, and the error flows back



Two inputs, four hidden units, one output. The task is XOR.

Forward: h = tanh(W1 x + b1), then p = sigmoid(W2 h + b2).

tanh (Session 11): from −1 to 1, slope 1 - h^2.

Backward: the chain rule passes the error back.

Backpropagation for the 2-4-1 network, in matrix algebra

Forward and loss $$h = \tanh(W_1 x + b_1), \quad p = \sigma(W_2 h + b_2), \quad L = -[\, y \ln p + (1-y)\ln(1-p)\,]$$

Backward $$\delta = p - y, \quad g_{W_2} = \delta\, h^T, \quad g_{b_2} = \delta, \quad \delta_h = (W_2^T \delta) \odot (1 - h^2), \quad g_{W_1} = \delta_h\, x^T, \quad g_{b_1} = \delta_h$$

Symbol	What it means	In our data
delta	The error signal at the output: p - y for a sigmoid with binary cross-entropy (Session 17).	−0.6112 in the example
delta_h	The error signal arriving at each hidden unit.	one value per unit
1 - h^2	The slope of tanh at each hidden unit (Session 11).	(0.990, 0.104, 0.961, 0.125)
odot	Multiply entry by entry.	unit by unit

Update with learning rate eta: each weight matrix and bias becomes W - eta × its gradient.

By hand: one forward and one backward step for the input (1, 2) with target 1

One illustrative example (not an XOR row), from the starting weights on the data slide. The notebook trains on many rows and averages their gradients.

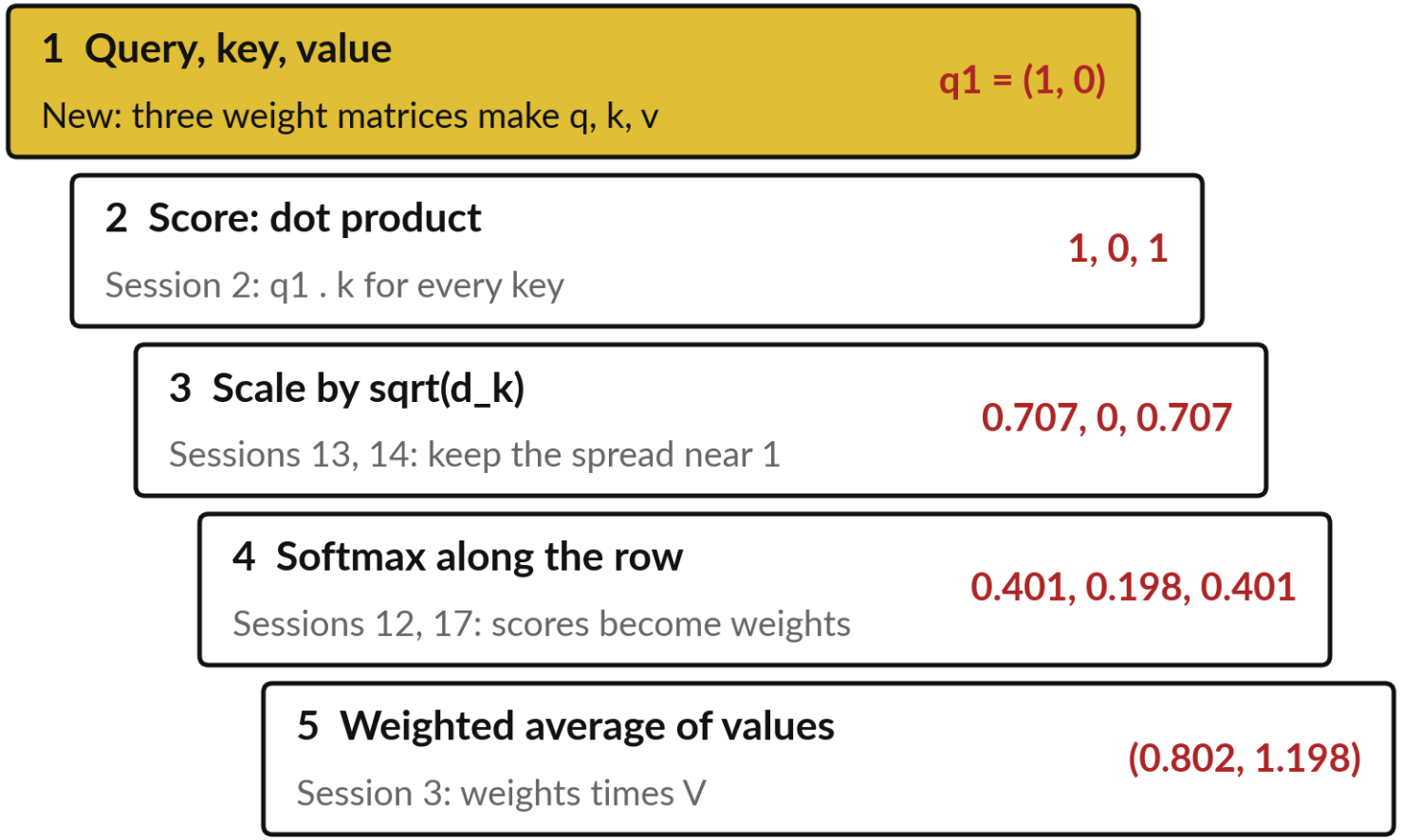
Forward	h = (−0.100, 0.947, 0.197, −0.935), z2 = −0.4522, p = 0.3888
Loss	-ln p = 0.9446 nats
Backward	delta = p - y = −0.6112. g_W2 = delta × h = (0.061, −0.579, −0.121, 0.572). First row of g_W1 = (−0.182, −0.363)
Update	W2 first weight: 0.3 - 0.5 × (0.061) = 0.270

After 5,000 gradient steps the toy network solves XOR (guided example)

Input	Target	Output before training	Output after training
(0, 0)	0	0.525	0.0002
(0, 1)	1	0.451	0.9989
(1, 0)	1	0.446	0.9992
(1, 1)	0	0.379	0.0011

The loss falls from 0.706 to 0.0008 nats. Learning rate 0.5.

Attention is new today: a ladder of five steps you already know



Each token asks a question (query), every token offers a label (key) and a message (value).

New today: q, k, v and how the steps fit together.

Known: dot products, scaling, softmax, matrix products.

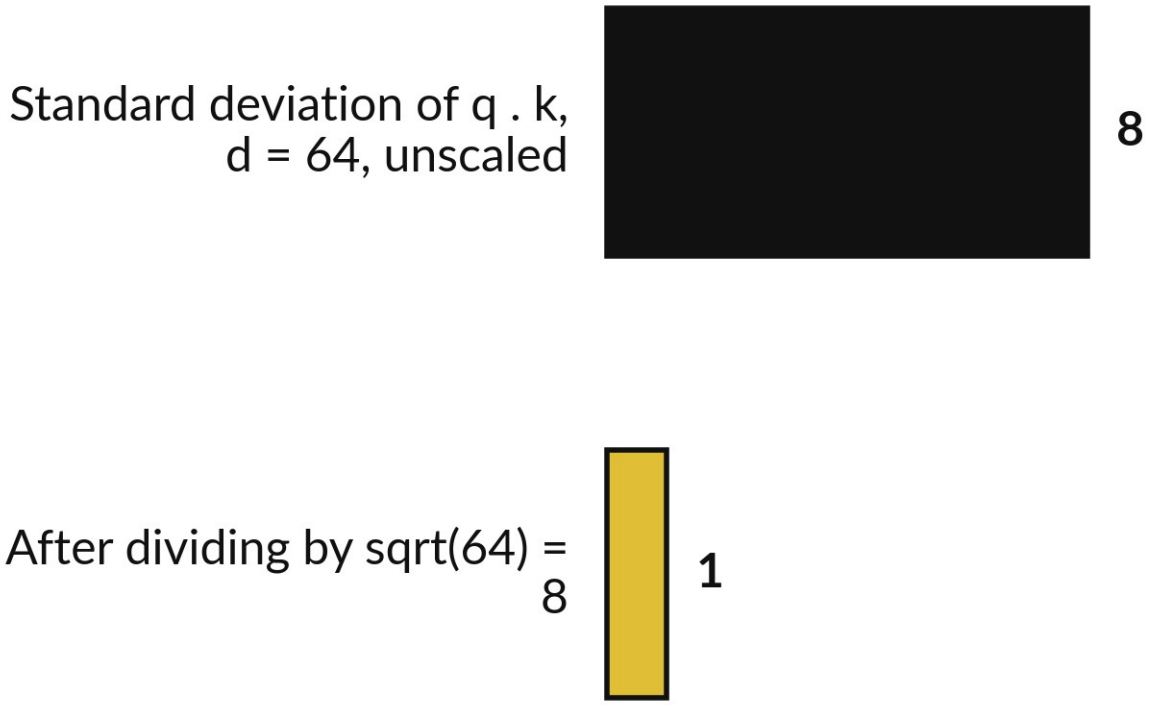
Red numbers: token 1 of the example.

By hand: attention for token 1, rung by rung

q1 = (1, 0). Keys k1 = (1, 1), k2 = (0, 1), k3 = (1, 2). Values v1 = (1, 0), v2 = (0, 2), v3 = (1, 2). d_k = 2.

Scores	q1 . k1 = 1, q1 . k2 = 0, q1 . k3 = 1
Scale	Divide by sqrt(2) = 1.414: 0.707, 0, 0.707
Softmax	e^0.707 = 2.028 and e^0 = 1. Sum 5.056. Weights 0.401, 0.198, 0.401.
Average	0.401 × (1, 0) plus 0.198 × (0, 2) plus 0.401 × (1, 2) = (0.802, 1.198)

Why divide by the square root of d: dot products of long vectors grow, and the softmax saturates



If the entries of q and k are independent with mean 0 and standard deviation 1, then q . k has mean 0 and standard deviation sqrt(d).

Saturate: when scores are large, the softmax puts almost all the weight on one token (nearly one-hot) and its gradient is tiny.

Dividing by sqrt(d_k) keeps the standard deviation near 1. The notebook simulates 2,000 pairs.

Build 5: all three tokens at once, as a grid of attention weights

rows = query tokens (asking) columns = key tokens (being read)

	Token 1	Token 2	Token 3	output = weights × V
Token 1	**0.401**	0.198	**0.401**	→ (0.80, 1.20)
Token 2	0.284	0.140	**0.576**	→ (0.86, 1.43)
Token 3	0.306	0.074	**0.620**	→ (0.93, 1.39)

Illustrative numbers. Token 3 reads 31 percent from token 1, 7 from token 2 and 62 from token 3.

Each row of weights adds to 1. The output is the weights times the value vectors.

The whole ladder in one formula, as published in 2017

Projections

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Symbol	What it means	In our data
X	One row per token: its embedding.	3 tokens, 2 numbers each
W_Q, W_K, W_V	Make queries, keys, values. Learned in a real model.	given here
Q K^T	All query-key dot products at once.	3 by 3
d_k; softmax	Key length; applied along each row.	2; row 1: 0.401, 0.198, 0.401

Vaswani et al. (2017). The W matrices are stored one column per output number, so they multiply on the right as they are.

By hand: scaled scores, weights and output for three tokens (d_k = 2)

Token	Scaled scores	Weights	Output
Token 1	0.707, 0.000, 0.707	0.401, 0.198, 0.401	0.802, 1.198
Token 2	1.414, 0.707, 2.121	0.284, 0.140, 0.576	0.860, 1.432
Token 3	2.121, 0.707, 2.828	0.306, 0.074, 0.620	0.926, 1.389

Illustrative. Scores = Q K^T / sqrt(2). Weights = softmax of each row. Output = weights times V.

Three slips that the self-checks will catch

The slip	The fix
No column of ones, so no intercept.	Add the column of ones to X.
PCA without centring the data first.	Subtract each column mean first.
Softmax along the wrong axis, or no sqrt(d_k).	Softmax along each row; divide scores by sqrt(d_k).

Each check names its build, so read the name first.

Read the five recipes before you write them

```
w = np.linalg.solve(X.T @ X, X.T @ y)
w = w - lr * X.T @ (sigmoid(X @ w) - y) / n
vals, vecs = np.linalg.eigh(Dc.T @ Dc / (n - 1))
gW1 = dh.T @ X
out = softmax_rows(Q @ K.T / np.sqrt(d)) @ V
```

In plain words

Build 1: the normal equation in one line.

Build 2: one gradient step of logistic regression.

Build 3: eigh gives eigenvalues smallest first, so take the last column. An eigenvector's sign is arbitrary.

Build 4: a gradient from backpropagation.

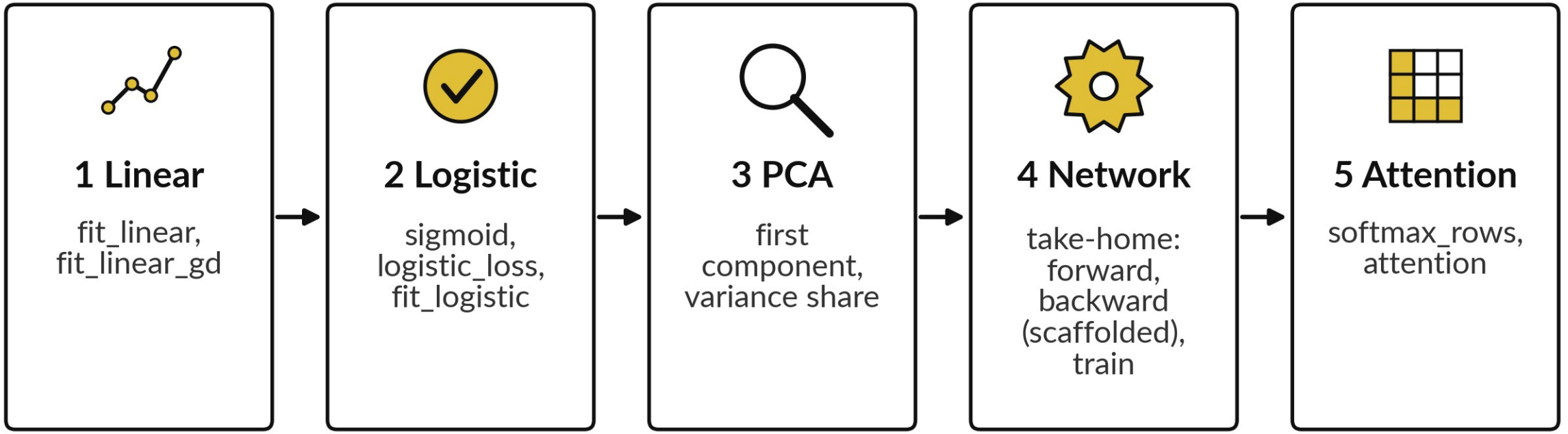
Build 5: the attention formula.

Spreadsheet: MMULT, MINVERSE, TRANSPOSE and SUMPRODUCT.

Part 2: your project

Close the slides. Open the notebook or the workbook.

Your project has five builds, and every self-check names its build



In class: Builds 1, 2, 3 and 5. At home: Build 4. Python: 41 checks, each on your data and a second dataset. Spreadsheet: a guided version.

How the self-checks work, and what counts as completing the project

	Step	What happens
🔑	**1 Student ID**	Type any text in the first cell. It only makes your own datasets, so your numbers are yours. It is not a test.
⚙	**2 Functions**	You write 12 functions with fixed names, from fit_linear to attention.
✔	**3 Self-checks**	Each one runs your function on your data and on a second dataset, and compares with a reference. You see OK or NOT YET; nothing is recorded.
🚩	**4 Completing**	Show your facilitator the last line of the notebook (show_checks) or the Check sheet of the guided workbook. Either counts.

No marks. Typed-in numbers show NOT YET, because the checks also use a second dataset.

Guided workbook: check your numbers against these (toy data)

	Build	What you should see
	**1 Linear regression**	b 1.12, w1 0.98; MSE 0.0213; 7 hours gives 7.98
	**2 Logistic regression**	after 5 steps w = (−0.0509, 0.1494), loss 0.6232
	**3 PCA**	eigenvalues 1.2840, 0.0491 (n - 1); first component (0.6779, 0.7352); share 0.9632
	**4 Neural network**	input (1, 2), target 1: p = 0.3888, loss 0.9446, delta = −0.6112, first g_W2 = 0.0609
	**5 Attention**	weights row 1: 0.401, 0.198, 0.401; output row 3: 0.926, 1.389

These toy numbers are for the guided workbook. The notebook shows no expected numbers. In numpy, eigh gives the eigenvalues smallest first.

Errors you will meet, what they mean and what to do

	What you see	What it means	What to do
	**TypeError: NoneType**	A function still says return None	Write your code in place of the placeholder
	**ValueError: shapes not aligned**	A matrix product has the wrong shapes	Print the shapes. Transpose one side
	**A check says NOT YET**	Your function disagrees with the reference on your data or on the second dataset	Call it on a tiny example and work it out by hand
	**Spreadsheet shows #VALUE!**	An array formula was not filled over the whole gold block	Select the block and enter it as an array formula

Project checklist: use it alone, then swap with a partner



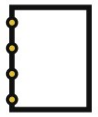
Every self-check says OK, or I can say what NOT YET still means.



I can explain each build in two sentences, with no formulas.

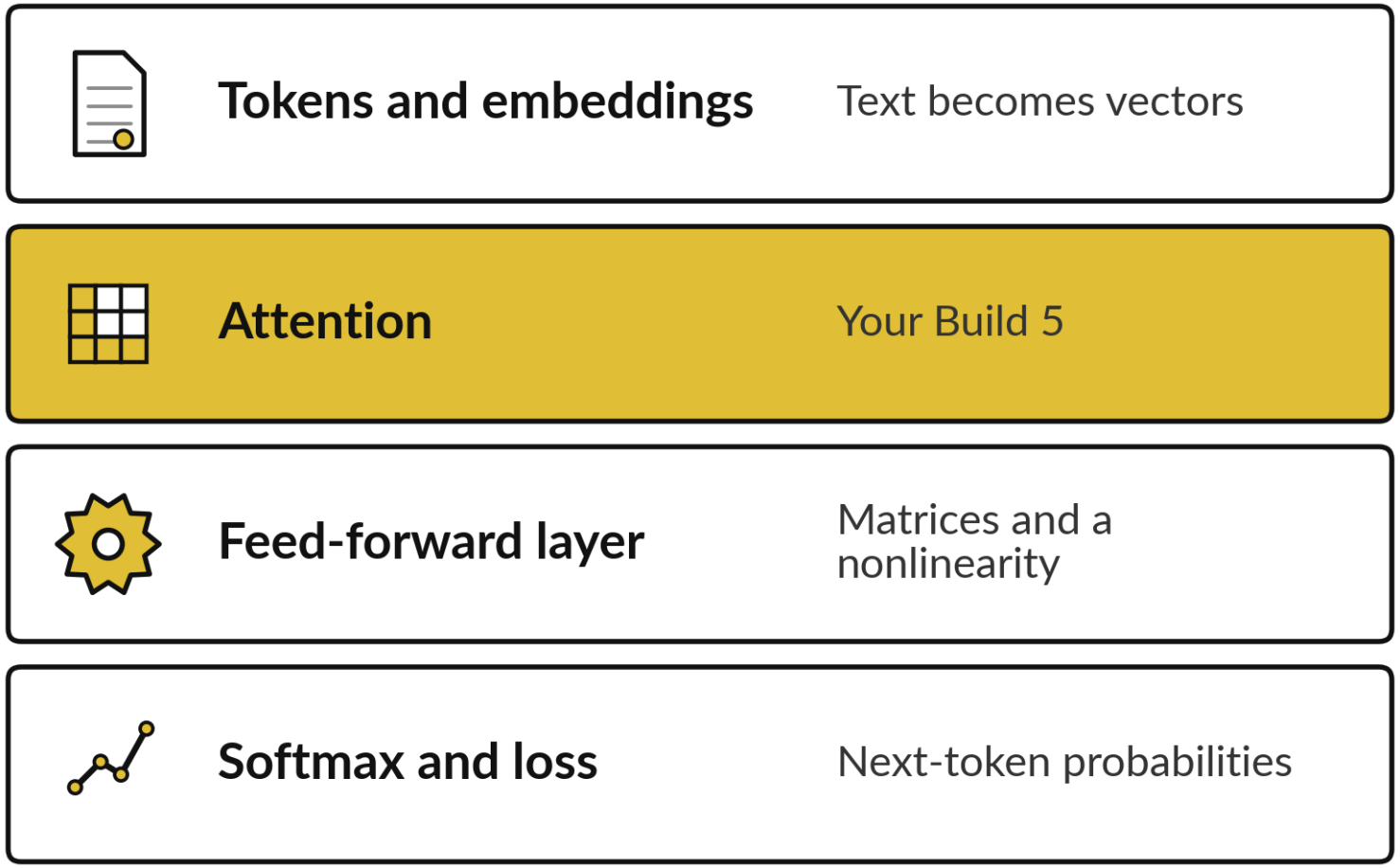


I changed one thing (learning rate, my own data, no scaling) and wrote what happened.



My notebook runs from the top with Restart and run all.

Where this appears in AI: attention is the core step of every layer of a published transformer



Published research describes transformer language models as many repeated layers of attention and feed-forward steps.

Feed-forward layer: a small network like Build 4 applied to each token. Nonlinearity: a curved function such as tanh between the matrices.

Open models such as Llama add a causal mask: a token cannot read later tokens.

Where this appears in AI: the five builds in published models

	Your build	Where the same idea appears
	**Linear regression**	Every layer multiplies by a weight matrix. Its weights are found by gradient descent rather than a direct formula, usually with a cross-entropy loss rather than squared error.
	**Logistic regression**	The last step of a classifier is a softmax of linear scores, trained with cross-entropy. This many-class version is called softmax regression.
	**PCA**	Used to inspect and compress embeddings. LoRA (Hu et al., 2021) learns a low-rank update to a weight matrix, the same low-rank idea.
	**Neural network**	Backpropagation runs in every training step of published models.
	**Attention**	The core step of the transformer, repeated in every layer.

Phrased as published research describes. No claim about the internals of any closed product.

Check an AI assistant's attention against your own numbers

5 minutes **Optional, with any assistant**

Give an assistant the three-token example: X, Wq, Wk, Wv from the data slide, d_k = 2. Ask for the attention weights.

Compare its row 1 with 0.401, 0.198, 0.401.

Ask if it divided by sqrt(d_k). Try without, and see which weights get sharper.

Never paste confidential data into an assistant.

You keep

A note: where the assistant agreed with your numbers, and where it did not.

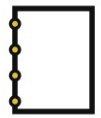
What to learn next: four steps



Machine Learning at KLAi: use these builds on real data, with scikit-learn.



A deep learning course, where PyTorch automatic differentiation replaces your hand-written backpropagation (title and dates to be confirmed by KLAi).



Read section 3.2 of Vaswani et al. (2017), "Attention Is All You Need", and match every symbol to your Build 5.



Rebuild the five builds from a blank notebook, then write your one-page formula sheet.

Exit check: three questions and their short answers

Question	Short answer
What does each row of the attention weights add up to, and why?	To 1, because a softmax is applied along each row.
Why is the attention score divided by the square root of d_k?	Large dot products make the softmax saturate: almost all the weight goes to one token and the gradient is tiny. The scaling keeps the standard deviation near 1.
In PCA, which eigenvector is the first principal component?	The one with the largest eigenvalue, the direction of largest variance.

Answer each in one sentence before you read the right-hand column.

Save your project and write your formula sheet

What you keep from today

Keep your finished project notebook, ideally with all self-checks OK, and write your one-page formula sheet: every formula named in plain words and in the industry term.

Our question today

Can five small builds, written from first principles, reproduce the core of what runs inside modern AI?

What to learn next

Machine Learning at KLAi, then deep learning (to be confirmed).

Take away

Your project
Self-checks OK

Formula sheet
One page

Next
Machine Learning at KLAi

Then
Deep learning